step_by_step_process

September 16, 2026

1 Protected Bidirectional Lung Parenchyma Segmentation

This notebook uses a **clean middle reference** to protect both lungs during propagation.

Main workflow:

1. Build a candidate mask for every slice using thresholding, border clearing, geometry-based table removal, largest-component selection, and hole filling.
2. Apply conservative peripheral cleanup and trachea removal to a validated bilateral reference slice.
3. Match candidate components in the next slice against the preceding clean mask.
4. Use seeded morphological reconstruction to produce a protected lung mask.
5. Retain the last valid reference across an empty or failed slice so that propagation can recover instead of failing cumulatively.
6. Remove residual trachea after propagation.
7. Repair boundary concavities such as the “apple-bite” artifact without using a nodule mask.
8. Use consensus nodule masks only to evaluate inclusion.

This technique is called **protected clean-seed bidirectional component propagation**.

2 Preparation

2.1 Import libraries

[1]:
```
import ast
import csv
from collections import OrderedDict
from pathlib import Path

from IPython.display import display
import matplotlib.pyplot as plt
from matplotlib.colors import to_rgb
from matplotlib.patches import Patch
import numpy as np
import pandas as pd
import pylidc as pl
import SimpleITK as sitk
from scipy import ndimage as ndi
from skimage.measure import label, regionprops
```

```
from skimage.morphology import disk
from skimage.segmentation import clear_border
from tqdm.auto import tqdm
```

```
/home/dityanugroho/miniconda3/envs/nodule_py310/lib/python3.10/site-
packages/pylidc/__init__.py:27: UserWarning: pkg_resources is deprecated as an
API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The
pkg_resources package is slated for removal as early as 2025-11-30. Refrain from
using this package or pin to Setuptools<81.
  import pkg_resources as _pr
/home/dityanugroho/miniconda3/envs/nodule_py310/lib/python3.10/site-
packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

2.2 Notebook settings and parameters

[2]:
```
%config InlineBackend.figure_format = "retina"

plt.rcParams.update({
    "figure.figsize": (12, 5),
    "figure.dpi": 100,
    "image.cmap": "gray",
    "image.interpolation": "nearest",
})

# Input data
# Use LIDC-IDRI-0077 to inspect the juxta-hilar regression; switch to
# LIDC-IDRI-0116 to inspect the wide-table regression.
LIDC_PATIENT_ID = "LIDC-IDRI-0077"
LNDB_MHD_PATH = Path("../000_dataset/lndb/data/LNDb-0002.mhd")

# Consensus nodule metadata and masks
LIDC_CLUSTER_METADATA_PATH = Path(
    "../000_dataset_v2/_lidc/000_metadata/003_cluster_metadata_cleaned_path.csv"
)
LIDC_CONSENSUS_MASK_DIR = Path("../000_dataset_v2/_lidc/004_mask_consensus_npy")
LNDB_CONSENSUS_METADATA_PATH = Path(
    "../000_dataset_v2/_lndb/000_metadata/004_consensus_clean_path.csv"
)
LNDB_CONSENSUS_MASK_DIR = Path("../000_dataset_v2/_lndb/004_mask_consensus_npy")

# Candidate-mask parameters
HU_THRESHOLD = -320
NUM_LARGEST_COMPONENTS = 4
TRACHEA_AREA_THRESHOLD = 0.0069
```

```
TRACHEA_CENTER_HALF_WIDTH_RATIO = 0.05
TRACHEA_MAX_Y_RATIO = 0.55

# Wide, flat lower-image table artifacts are removed from every candidate.
TABLE_LOWER_CENTER_Y_RATIO = 0.70
TABLE_MIN_WIDTH_RATIO = 0.40
TABLE_MAX_HEIGHT_WIDTH_RATIO = 0.35

# Additional small-peripheral cleanup is restricted to the reference slice.
TABLE_COMPONENT_AREA_THRESHOLD = 0.02
TABLE_MIN_Y_RATIO = 0.30
TABLE_MAX_Y_RATIO = 0.60

# Try a strict overlap first, then enlarge the search area gradually.
REFERENCE_DILATION_SCHEDULE = (3, 6, 10, 15)
MIN_COMPONENT_OVERLAP_PIXELS = 10
BOUNDARY_REPAIR_RADIUS = 16

# Use None to select the middle slice automatically.
REFERENCE_SLICE_INDICES = {
    "LIDC-IDRI": None,
    "LNDb": None,
}

MEDIAN_FILTER_SIZE = (1, 3, 3)
LUNG_WINDOW = (-1400, 200)
WINDOW_LEVEL = -600.0
WINDOW_WIDTH = 1600.0
NUM_QC_SLICES = 9
NODULE_SLICES_PER_FIGURE = 9
```

3 Step 1: Load CT Volumes

All volumes use the `[slice, height, width]` layout and retain their original Hounsfield Unit values.

3.1 LIDC-IDRI DICOM to NumPy

[3]:
```
scan = pl.query(pl.Scan).filter(
    pl.Scan.patient_id == LIDC_PATIENT_ID
).first()

if scan is None:
    raise ValueError(f"Patient not found: {LIDC_PATIENT_ID}")

# pylidc returns [height, width, slice].
lidc_volume = np.transpose(scan.to_volume(), (2, 0, 1)).astype(np.int16)
```

```
print(f"LIDC-IDRI shape={lidc_volume.shape}, dtype={lidc_volume.dtype}")
```

```
Loading dicom files … This may take a moment.
LIDC-IDRI shape=(130, 512, 512), dtype=int16
```

3.2 LNDb MHD to NumPy

[4]:
```
lndb_image = sitk.ReadImage(str(LNDB_MHD_PATH))
# SimpleITK returns [slice, height, width].
lndb_volume = sitk.GetArrayFromImage(lndb_image).astype(np.int16)
print(f"LNDb shape={lndb_volume.shape}, dtype={lndb_volume.dtype}")
```

```
LNDb shape=(263, 512, 512), dtype=int16
```

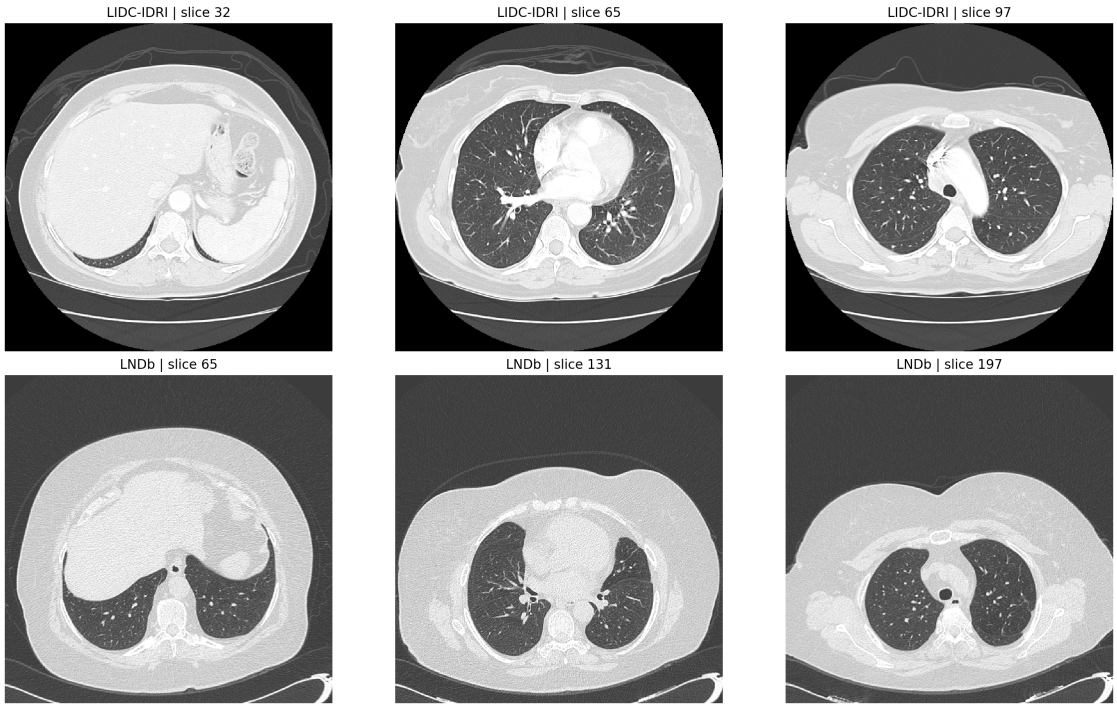
[5]:
```
ct_volumes = OrderedDict([
    ("LIDC-IDRI", lidc_volume),
    ("LNDb", lndb_volume),
])
```

3.3 CT overview

[6]:
```
fig, axes = plt.subplots(2, 3, figsize=(15, 9), constrained_layout=True)

for row, (dataset_name, volume) in enumerate(ct_volumes.items()):
    indices = [volume.shape[0] // 4, volume.shape[0] // 2, 3 * volume.shape[0] /
 ↪/ 4]
    for axis, slice_index in zip(axes[row], indices):
        axis.imshow(volume[slice_index], vmin=LUNG_WINDOW[0],␣
 ↪vmax=LUNG_WINDOW[1])
        axis.set_title(f"{dataset_name} | slice {slice_index}")
        axis.axis("off")

plt.show()
```



4 Step 2: High-Sensitivity Candidate Mask for Every Slice

Every slice undergoes:

`threshold → remove border → remove wide/flat table → keep largest components → fill holes`

Only the high-specificity geometric table rule is applied to every candidate. Trachea removal and small-peripheral cleanup remain restricted to the reference/final stages so that a small lung region is not discarded prematurely.

4.1 Segmentation operations

[7]:
```python
def threshold_lung(slice_image):
    return slice_image < HU_THRESHOLD


def remove_border_objects(mask):
    return clear_border(mask)


def keep_largest_components(mask, number=NUM_LARGEST_COMPONENTS):
    labeled_mask = label(mask)
    regions = sorted(
        regionprops(labeled_mask),
```

```
        key=lambda region: region.area,
        reverse=True,
    )

    output = np.zeros_like(mask, dtype=bool)
    for region in regions[:number]:
        output[labeled_mask == region.label] = True
    return output


def fill_holes(mask):
    return ndi.binary_fill_holes(mask)


def remove_wide_flat_table(mask):
    # High-specificity rule that also catches a table larger than 2% of the␣
 ↪image.
    labeled_mask = label(mask)
    output = mask.copy()
    height, width = mask.shape

    for region in regionprops(labeled_mask):
        min_y, min_x, max_y, max_x = region.bbox
        component_height = max_y - min_y
        component_width = max_x - min_x
        height_width_ratio = component_height / max(component_width, 1)
        is_lower = region.centroid[0] > TABLE_LOWER_CENTER_Y_RATIO * height
        is_wide = component_width > TABLE_MIN_WIDTH_RATIO * width
        is_flat = height_width_ratio < TABLE_MAX_HEIGHT_WIDTH_RATIO

        if is_lower and is_wide and is_flat:
            output[labeled_mask == region.label] = False

    return output


def remove_trachea(mask):
    # Used only for the middle reference and residual cleanup.
    labeled_mask = label(mask)
    output = mask.copy()
    height, width = mask.shape
    image_area = height * width

    for region in regionprops(labeled_mask):
        center_y, center_x = region.centroid
        is_small = region.area / image_area < TRACHEA_AREA_THRESHOLD
        is_strictly_central = (
```

```
            abs(center_x - width / 2)
            < TRACHEA_CENTER_HALF_WIDTH_RATIO * width
        )
        is_anterior = center_y < TRACHEA_MAX_Y_RATIO * height

        if is_small and is_strictly_central and is_anterior:
            output[labeled_mask == region.label] = False

    return output


def remove_table(mask):
    # Geometric table removal is safe for all slices; the additional
    # small-peripheral rule is used only while cleaning the reference.
    output = remove_wide_flat_table(mask)
    labeled_mask = label(output)
    height, width = mask.shape
    image_area = height * width

    for region in regionprops(labeled_mask):
        center_y = region.centroid[0]
        is_small = region.area / image_area < TABLE_COMPONENT_AREA_THRESHOLD
        is_peripheral = not (
            TABLE_MIN_Y_RATIO * height
            <= center_y
            <= TABLE_MAX_Y_RATIO * height
        )
        if is_small and is_peripheral:
            output[labeled_mask == region.label] = False

    return output
```

4.2 Collect every intermediate stage

[8]:
```
CANDIDATE_STAGE_FUNCTIONS = OrderedDict([
    ("1. Threshold HU", threshold_lung),
    ("2. Remove border objects", remove_border_objects),
    ("3. Remove wide/flat table", remove_wide_flat_table),
    ("4. Keep largest components", keep_largest_components),
    ("5. Fill holes", fill_holes),
])


def collect_candidate_stages(slice_image):
    stages = OrderedDict()
    mask = slice_image
```

```
    for stage_name, operation in CANDIDATE_STAGE_FUNCTIONS.items():
        mask = operation(mask)
        stages[stage_name] = np.asarray(mask, dtype=bool)

    return stages


def segment_candidate_slice(slice_image):
    return next(reversed(collect_candidate_stages(slice_image).values()))
```

4.3 Visualize every candidate-mask stage

[9]:
```
for dataset_name, volume in ct_volumes.items():
    slice_index = volume.shape[0] // 2
    slice_image = volume[slice_index]
    stages = collect_candidate_stages(slice_image)

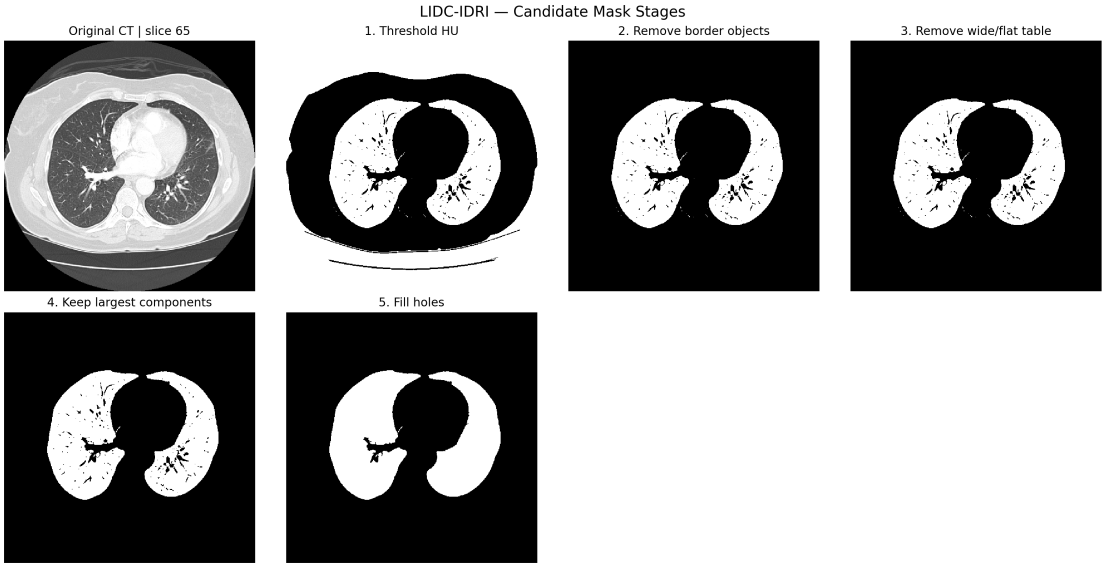
    fig, axes = plt.subplots(2, 4, figsize=(16, 8), constrained_layout=True)
    axes = axes.ravel()

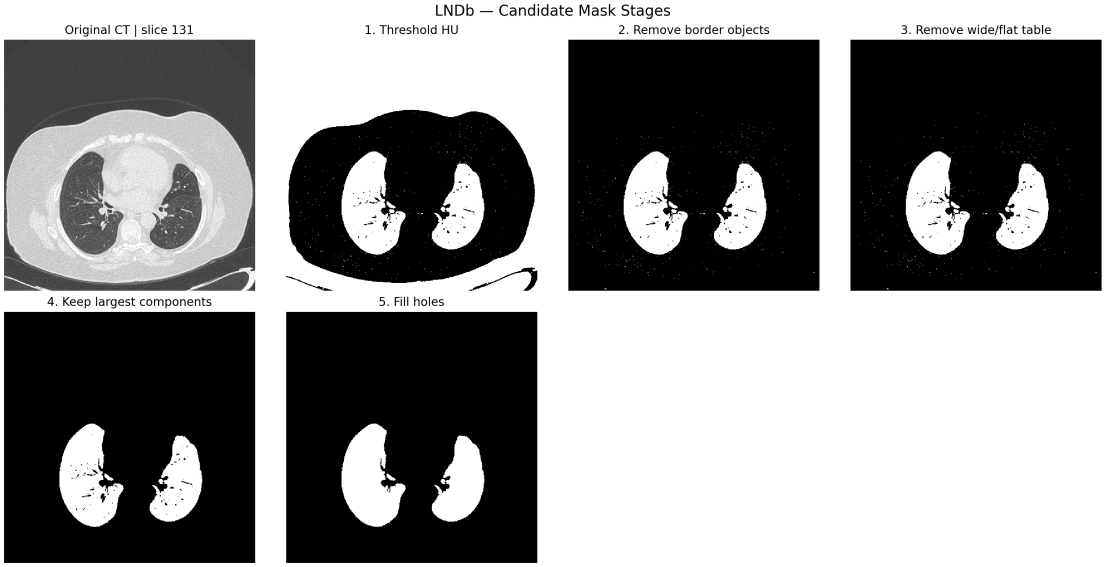
    axes[0].imshow(slice_image, vmin=LUNG_WINDOW[0], vmax=LUNG_WINDOW[1])
    axes[0].set_title(f"Original CT | slice {slice_index}")

    for axis, (stage_name, stage_mask) in zip(axes[1:], stages.items()):
        axis.imshow(stage_mask, vmin=0, vmax=1)
        axis.set_title(stage_name)

    for axis in axes:
        axis.axis("off")

    fig.suptitle(f"{dataset_name} - Candidate Mask Stages", fontsize=15)
    plt.show()
```





4.4 Process all slices

[10]:
```
def build_candidate_volume(volume, description):
    candidate_volume = np.zeros(volume.shape, dtype=bool)

    for slice_index in tqdm(range(len(volume)), desc=description):
        candidate_volume[slice_index] = segment_candidate_slice(
            volume[slice_index]
        )
```

```
    return candidate_volume


candidate_volumes = OrderedDict()
for dataset_name, volume in ct_volumes.items():
    candidate_volumes[dataset_name] = build_candidate_volume(
        volume,
        description=f"Candidate masks - {dataset_name}",
    )
```

```
Candidate masks - LIDC-IDRI: 100%|      | 130/130 [00:02<00:00, 49.56it/s]
Candidate masks - LNDb: 100%|      | 263/263 [00:17<00:00, 15.47it/s]
```

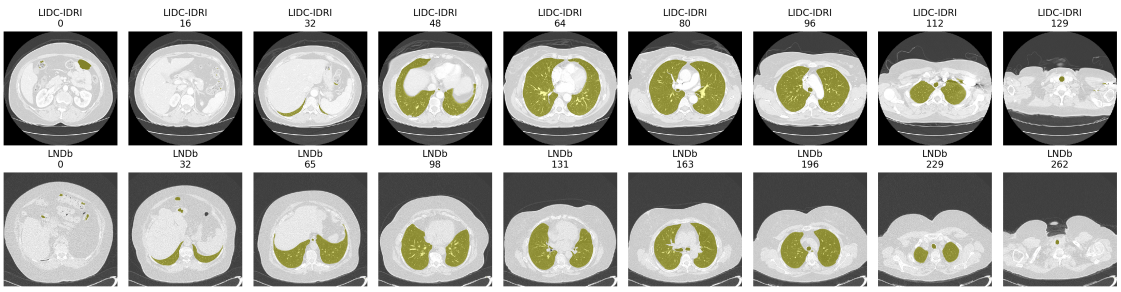
4.5 Candidate-mask quality control

[11]:
```
fig, axes = plt.subplots(2, NUM_QC_SLICES, figsize=(20, 5),␣
↪constrained_layout=True)

for row, (dataset_name, volume) in enumerate(ct_volumes.items()):
    indices = np.linspace(0, len(volume) - 1, NUM_QC_SLICES, dtype=int)
    candidates = candidate_volumes[dataset_name]

    for axis, slice_index in zip(axes[row], indices):
        axis.imshow(volume[slice_index], vmin=LUNG_WINDOW[0],␣
↪vmax=LUNG_WINDOW[1])
        axis.imshow(
            np.ma.masked_where(~candidates[slice_index],␣
↪candidates[slice_index]),
            cmap="autumn",
            alpha=0.35,
            vmin=0,
            vmax=1,
        )
        axis.set_title(f"{dataset_name}\n{slice_index}")
        axis.axis("off")

plt.show()
```



5 Step 3: Create a Clean Bilateral Reference Mask

Reference candidates, starting from the middle slice, undergo:

`remove table → remove trachea → keep two lung components → fill holes`

Automatic selection prefers the nearest slice containing plausible components on both sides of the image. This prevents a table or another single component from silently becoming the second lung seed. Use `REFERENCE_SLICE_INDICES` to select a reference slice manually when necessary.

[12]:
```python
def clean_reference_mask(candidate_mask):
    cleaned = remove_table(candidate_mask)
    cleaned = remove_trachea(cleaned)
    cleaned = keep_largest_components(cleaned, number=2)
    return ndi.binary_fill_holes(cleaned).astype(bool)


def has_bilateral_lung_components(mask):
    # A preferred reference has substantial components on both image halves.
    height, width = mask.shape
    minimum_area = 0.01 * height * width
    regions = [
        region for region in regionprops(label(mask))
        if region.area >= minimum_area
    ]
    center_x_values = [region.centroid[1] for region in regions]
    return (
        any(center_x < width / 2 for center_x in center_x_values)
        and any(center_x > width / 2 for center_x in center_x_values)
    )


def select_reference_index(candidate_volume, requested_index=None):
    depth = len(candidate_volume)

    if requested_index is not None:
        reference_index = int(requested_index)
        if not 0 <= reference_index < depth:
            raise IndexError(f"Reference index must be in [0, {depth - 1}]")
        cleaned = clean_reference_mask(candidate_volume[reference_index])
        if not cleaned.any():
            raise ValueError("The cleaned reference mask is empty.")
        return reference_index

    middle_index = depth // 2
```

```
    search_order = sorted(
        range(depth),
        key=lambda index: abs(index - middle_index),
    )
    nearest_valid_index = None

    for index in search_order:
        cleaned = clean_reference_mask(candidate_volume[index])
        if not cleaned.any():
            continue
        if nearest_valid_index is None:
            nearest_valid_index = index
        if has_bilateral_lung_components(cleaned):
            return index

    if nearest_valid_index is not None:
        return nearest_valid_index
    raise ValueError("No non-empty clean reference mask is available.")


reference_indices = OrderedDict()
clean_reference_masks = OrderedDict()

for dataset_name, candidates in candidate_volumes.items():
    reference_index = select_reference_index(
        candidates,
        REFERENCE_SLICE_INDICES[dataset_name],
    )
    clean_mask = clean_reference_mask(candidates[reference_index])
    if not clean_mask.any():
        raise ValueError(
            f"{dataset_name}: the clean reference mask on slice "
            f"{reference_index} is empty. Select another reference slice."
        )

    reference_indices[dataset_name] = reference_index
    clean_reference_masks[dataset_name] = clean_mask
    print(
        f"{dataset_name}: reference slice={reference_index}, "
        f"candidate area={candidates[reference_index].sum()}, "
        f"clean area={clean_mask.sum()}"
    )
```

```
LIDC-IDRI: reference slice=65, candidate area=49441, clean area=49409
LNDb: reference slice=131, candidate area=29041, clean area=28829
```

5.1 Reference-slice cleaning visualization

The middle panel shows the candidate mask before cleaning. The right panel shows the clean reference mask that initializes propagation.

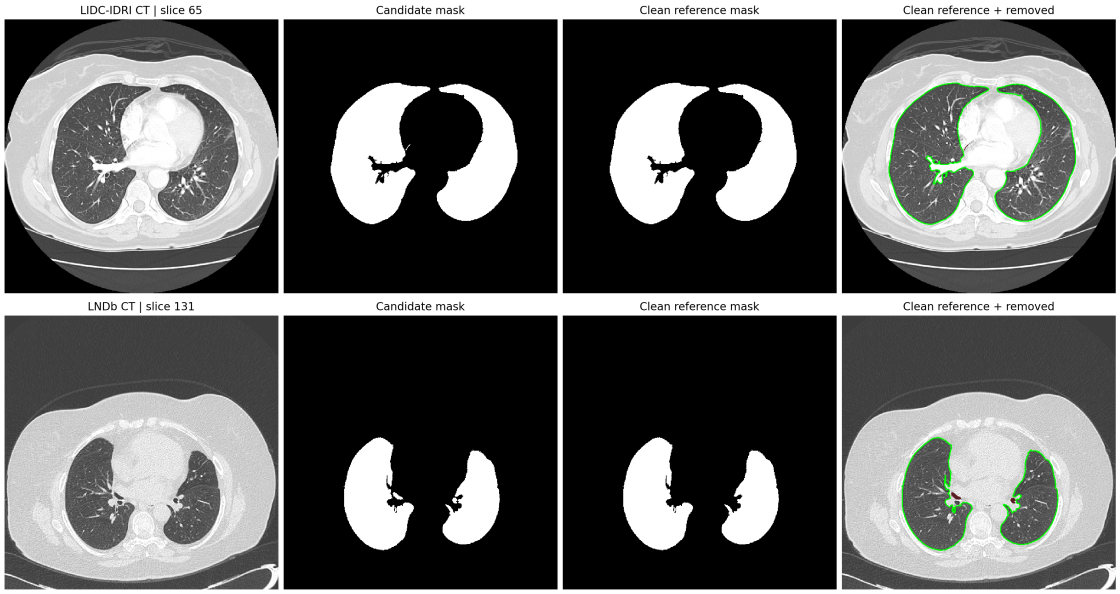
[13]:
```
fig, axes = plt.subplots(2, 4, figsize=(17, 9), constrained_layout=True)

for row, (dataset_name, volume) in enumerate(ct_volumes.items()):
    index = reference_indices[dataset_name]
    candidate = candidate_volumes[dataset_name][index]
    clean_mask = clean_reference_masks[dataset_name]
    removed = candidate & ~clean_mask

    axes[row, 0].imshow(volume[index], vmin=LUNG_WINDOW[0], vmax=LUNG_WINDOW[1])
    axes[row, 0].set_title(f"{dataset_name} CT | slice {index}")
    axes[row, 1].imshow(candidate, vmin=0, vmax=1)
    axes[row, 1].set_title("Candidate mask")
    axes[row, 2].imshow(clean_mask, vmin=0, vmax=1)
    axes[row, 2].set_title("Clean reference mask")
    axes[row, 3].imshow(volume[index], vmin=LUNG_WINDOW[0], vmax=LUNG_WINDOW[1])
    if clean_mask.any():
        axes[row, 3].contour(clean_mask, levels=[0.5], colors="lime",␣
 ↪linewidths=1.2)
    axes[row, 3].imshow(
        np.ma.masked_where(~removed, removed),
        cmap="Reds", alpha=0.6, vmin=0, vmax=1,
    )
    axes[row, 3].set_title("Clean reference + removed")

    for axis in axes[row]:
        axis.axis("off")

plt.show()
```



6 Step 4: Protected Bidirectional Component Propagation

For each new slice:

1. Gradually dilate the preceding clean mask.
2. Check every connected candidate component against the reference support.
3. Retain only components with sufficient overlap as matched candidates.
4. Use overlapping pixels as seeds for morphological reconstruction.
5. Use a successful result as both the protected lung mask and the reference for the next slice.
6. If a slice is empty or overlap fails, keep the last valid reference for the following slice.

The complete candidate mask is never used directly as a fallback. Retaining the last valid reference prevents one missing slice from forcing every subsequent propagation result to remain empty.

6.1 One propagation step

[14]:
```python
def match_candidate_components(candidate_mask, reference_support):
    labeled_candidate = label(candidate_mask)
    matched_candidate = np.zeros_like(candidate_mask, dtype=bool)
    seed = np.zeros_like(candidate_mask, dtype=bool)

    for region in regionprops(labeled_candidate):
        component = labeled_candidate == region.label
        component_seed = component & reference_support

        if component_seed.sum() >= MIN_COMPONENT_OVERLAP_PIXELS:
            matched_candidate |= component
            seed |= component_seed
```

```
    return matched_candidate, seed


def propagate_one_slice(candidate_mask, reference_mask):
    last_support = np.zeros_like(reference_mask, dtype=bool)
    last_matched = np.zeros_like(candidate_mask, dtype=bool)

    for dilation_iterations in REFERENCE_DILATION_SCHEDULE:
        reference_support = ndi.binary_dilation(
            reference_mask,
            iterations=dilation_iterations,
        )
        matched_candidate, seed = match_candidate_components(
            candidate_mask,
            reference_support,
        )
        last_support = reference_support
        last_matched = matched_candidate

        if seed.any():
            propagated_mask = ndi.binary_propagation(
                seed,
                mask=matched_candidate,
            )
            propagated_mask = ndi.binary_fill_holes(propagated_mask)
            return (
                propagated_mask.astype(bool),
                matched_candidate,
                seed,
                reference_support,
                dilation_iterations,
                False,
            )

    empty_seed = np.zeros_like(candidate_mask, dtype=bool)
    empty_mask = np.zeros_like(candidate_mask, dtype=bool)
    return empty_mask, last_matched, empty_seed, last_support, None, True
```

6.2 Propagate from the reference to both directions

[15]:
```
def propagate_bidirectionally(candidate_volume, reference_index,␣
↪clean_reference):
    protected = np.zeros_like(candidate_volume, dtype=bool)
    protected[reference_index] = clean_reference
    failed_overlap_indices = []
```

```
    last_valid_reference = clean_reference
    for z in range(reference_index - 1, -1, -1):
        propagated, _, _, _, _, failed = propagate_one_slice(
            candidate_volume[z],
            last_valid_reference,
        )
        protected[z] = propagated
        if failed and candidate_volume[z].any():
            failed_overlap_indices.append(z)
        if not failed:
            last_valid_reference = propagated

    last_valid_reference = clean_reference
    for z in range(reference_index + 1, len(candidate_volume)):
        propagated, _, _, _, _, failed = propagate_one_slice(
            candidate_volume[z],
            last_valid_reference,
        )
        protected[z] = propagated
        if failed and candidate_volume[z].any():
            failed_overlap_indices.append(z)
        if not failed:
            last_valid_reference = propagated

    return protected, sorted(failed_overlap_indices)


protected_lung_volumes = OrderedDict()
failed_overlap_slices = OrderedDict()

for dataset_name, candidates in candidate_volumes.items():
    protected_lung_volumes[dataset_name], failed_overlap_slices[dataset_name] =␣
 ↪(
        propagate_bidirectionally(
            candidates,
            reference_indices[dataset_name],
            clean_reference_masks[dataset_name],
        )
    )
    print(
        f"{dataset_name}: failed-overlap slices "
        f"{failed_overlap_slices[dataset_name]}"
    )
```

```
LIDC-IDRI: failed-overlap slices [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13,
14, 15, 16, 17, 18, 19, 20, 21, 22, 23]
LNDb: failed-overlap slices [0]
```

7 Step 5: Residual Trachea Removal and Boundary Repair

Residual trachea is removed only after both lungs have been protected through propagation. Trachea detection uses a narrow central band so that a small right lung is not removed.

Morphological closing with radius 16 is then applied **per connected component** to repair boundary concavities such as the "apple-bite" artifact. Radius 16 captures nearly all of the juxta-hilar nodule in LIDC-IDRI-0077 while remaining more conservative than radius 18 or a convex hull. Consensus nodule masks are not used in this repair.

[16]:
```
def remove_residual_trachea(mask):
    return remove_trachea(mask)


def repair_lung_boundary(mask, radius=BOUNDARY_REPAIR_RADIUS):
    labeled_mask = label(mask)
    repaired_mask = np.zeros_like(mask, dtype=bool)
    structure = disk(radius)

    # Process components separately so the left and right lungs do not merge.
    for region in regionprops(labeled_mask):
        component = labeled_mask == region.label
        repaired_component = ndi.binary_closing(
            component,
            structure=structure,
        )
        repaired_component = ndi.binary_fill_holes(repaired_component)
        repaired_mask |= repaired_component

    return repaired_mask.astype(bool)


trachea_cleaned_volumes = OrderedDict()
propagated_volumes = OrderedDict()  # Final lung masks used downstream.

for dataset_name, protected_volume in protected_lung_volumes.items():
    trachea_cleaned = np.zeros_like(protected_volume, dtype=bool)
    final_volume = np.zeros_like(protected_volume, dtype=bool)

    for z in tqdm(range(len(protected_volume)), desc=f"Boundary repair -␣
 ↪{dataset_name}"):
        trachea_cleaned[z] = remove_residual_trachea(protected_volume[z])
        final_volume[z] = repair_lung_boundary(trachea_cleaned[z])

    trachea_cleaned_volumes[dataset_name] = trachea_cleaned
    propagated_volumes[dataset_name] = final_volume
```

```
Boundary repair - LIDC-IDRI: 100%|      | 130/130 [00:38<00:00,  3.34it/s]
Boundary repair - LNDb: 100%|      | 263/263 [01:45<00:00,  2.50it/s]
```

7.1 Protected, trachea-cleaned, and boundary-repaired masks

[ ]:
```
INSPECTION_SLICES = {
    "LIDC-IDRI": [0, 32, 67, 100, 120],
    "LNDb": [32, 43, 44, 45, 229],
}

for dataset_name, indices in INSPECTION_SLICES.items():
    volume = ct_volumes[dataset_name]
    rows = len(indices)
    fig, axes = plt.subplots(
        rows, 4,
        figsize=(14, 3.3 * rows),
        constrained_layout=True,
        squeeze=False,
    )

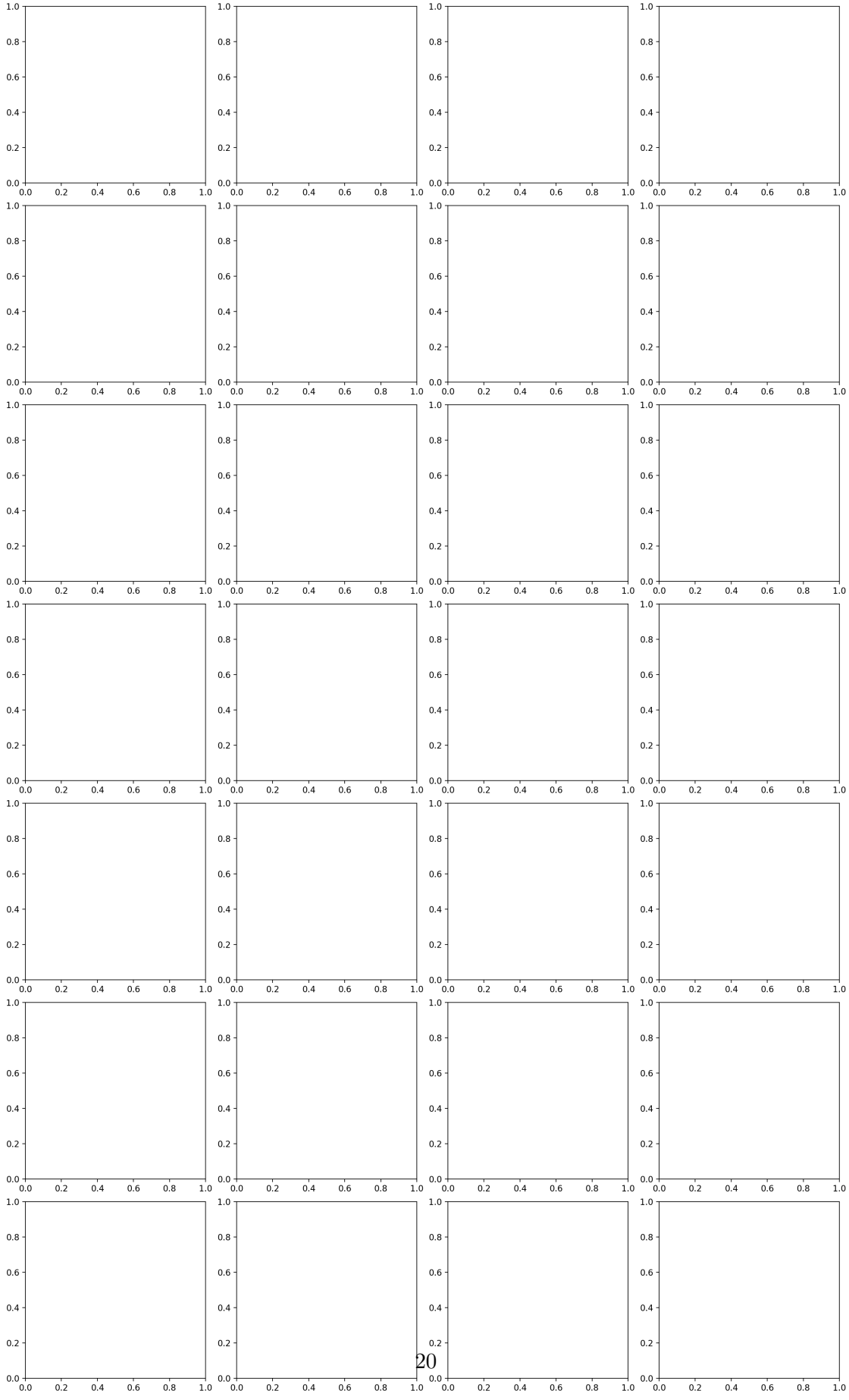
    for row, z in enumerate(indices):
        stages = [
            ("CT", None),
            ("Protected", protected_lung_volumes[dataset_name][z]),
            ("Residual trachea removed",␣
 ↪trachea_cleaned_volumes[dataset_name][z]),
            ("Boundary repaired", propagated_volumes[dataset_name][z]),
        ]
        for column, (title, stage_mask) in enumerate(stages):
            axes[row, column].imshow(
                volume[z], vmin=LUNG_WINDOW[0], vmax=LUNG_WINDOW[1]
            )
            if stage_mask is not None and stage_mask.any():
                axes[row, column].contour(
                    stage_mask, levels=[0.5], colors="lime", linewidths=1.2
                )
            axes[row, column].set_title(f"{title} | slice {z}")
            axes[row, column].axis("off")

    fig.suptitle(f"{dataset_name} - Post-propagation Stages", fontsize=16)
    plt.show()
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[17], line 19
      9 fig, axes = plt.subplots(
     10     rows, 4,
     11     figsize=(14, 3.3 * rows),
     12     constrained_layout=True,
     13     squeeze=False,
```

```
    14 )
    16 for row, z in enumerate(indices):
    17     stages = [
    18         ("CT", None),
---> 19         ("Protected", protected_lung_volumes[dataset_name][z]),
    20         ("Residual trachea removed",␣
 ↪trachea_cleaned_volumes[dataset_name][z]),
    21         ("Boundary repaired", propagated_volumes[dataset_name][z]),
    22     ]
    23     for column, (title, stage_mask) in enumerate(stages):
    24         axes[row, column].imshow(
    25             volume[z], vmin=LUNG_WINDOW[0], vmax=LUNG_WINDOW[1]
    26         )

IndexError: index 179 is out of bounds for axis 0 with size 130
```


0.0 0.2 0.4 0.6 0.8 1.0

7.2 Visualize the propagation mechanism

The following visualization uses the protected mask before residual-trachea removal and boundary repair, making the component-matching mechanism directly observable.

[ ]:
```
def propagation_diagnostics(
    candidates, protected, index, reference_index, clean_reference
):
    if index == reference_index:
        reference = clean_reference
        support = reference
        matched = reference
        seed = reference
        direction = "clean reference"
        dilation_used = 0
    elif index < reference_index:
        reference = protected[index + 1]
        _, matched, seed, support, dilation_used, _ = propagate_one_slice(
            candidates[index], reference
        )
        direction = f"reference={index + 1}"
    else:
        reference = protected[index - 1]
        _, matched, seed, support, dilation_used, _ = propagate_one_slice(
            candidates[index], reference
        )
        direction = f"reference={index - 1}"
    return reference, support, matched, seed, direction, dilation_used


for dataset_name, volume in ct_volumes.items():
    reference_index = reference_indices[dataset_name]
    candidates = candidate_volumes[dataset_name]
    protected = protected_lung_volumes[dataset_name]
    offset = max(1, len(volume) // 8)
    selected_indices = sorted(set([
        max(0, reference_index - offset),
        reference_index,
        min(len(volume) - 1, reference_index + offset),
    ]))

    fig, axes = plt.subplots(
        len(selected_indices), 6,
        figsize=(21, 4 * len(selected_indices)),
        constrained_layout=True,
```

```
        squeeze=False,
    )

    for row, index in enumerate(selected_indices):
        reference, support, matched, seed, direction, dilation_used = (
            propagation_diagnostics(
                candidates, protected, index, reference_index,
                clean_reference_masks[dataset_name],
            )
        )

        panels = [
            ("CT", volume[index]),
            ("Candidate", candidates[index]),
            (f"Reference support\n{direction}; d={dilation_used}", support),
            ("Matched components", matched),
            ("Seed", seed),
            ("Protected lung", protected[index]),
        ]
        for column, (title, image) in enumerate(panels):
            axes[row, column].imshow(
                image,
                vmin=LUNG_WINDOW[0] if column == 0 else 0,
                vmax=LUNG_WINDOW[1] if column == 0 else 1,
            )
            axes[row, column].set_title(title)
            axes[row, column].axis("off")

    fig.suptitle(f"{dataset_name} - Protected Component Propagation",␣
 ↪fontsize=16)
    plt.show()
```

8 Step 6: Final Segmentation Visualization

8.1 Final axial montage

[ ]:
```
fig, axes = plt.subplots(2, NUM_QC_SLICES, figsize=(20, 5),␣
 ↪constrained_layout=True)

for row, (dataset_name, volume) in enumerate(ct_volumes.items()):
    indices = np.linspace(0, len(volume) - 1, NUM_QC_SLICES, dtype=int)
    propagated = propagated_volumes[dataset_name]

    for axis, index in zip(axes[row], indices):
        axis.imshow(volume[index], vmin=LUNG_WINDOW[0], vmax=LUNG_WINDOW[1])
        if propagated[index].any():
```

```
            axis.contour(propagated[index], levels=[0.5], colors="lime",␣
 ↪linewidths=1)
        axis.set_title(f"{dataset_name}\n{index}")
        axis.axis("off")

plt.show()
```

8.2 Candidate versus propagated mask

[ ]:
```
for dataset_name, volume in ct_volumes.items():
    candidates = candidate_volumes[dataset_name]
    propagated = propagated_volumes[dataset_name]
    reference_index = reference_indices[dataset_name]
    indices = np.linspace(0, len(volume) - 1, 6, dtype=int)

    fig, axes = plt.subplots(2, len(indices), figsize=(18, 7),␣
 ↪constrained_layout=True)

    for column, index in enumerate(indices):
        rejected = candidates[index] & ~propagated[index]

        axes[0, column].imshow(volume[index], vmin=LUNG_WINDOW[0],␣
 ↪vmax=LUNG_WINDOW[1])
        axes[0, column].imshow(
            np.ma.masked_where(~candidates[index], candidates[index]),
            cmap="autumn", alpha=0.35, vmin=0, vmax=1,
        )
        axes[0, column].set_title(f"Candidate | {index}")

        axes[1, column].imshow(volume[index], vmin=LUNG_WINDOW[0],␣
 ↪vmax=LUNG_WINDOW[1])
        axes[1, column].imshow(
            np.ma.masked_where(~propagated[index], propagated[index]),
            cmap="Greens", alpha=0.35, vmin=0, vmax=1,
        )
        axes[1, column].imshow(
            np.ma.masked_where(~rejected, rejected),
            cmap="Reds", alpha=0.65, vmin=0, vmax=1,
        )
        axes[1, column].set_title(f"Final + rejected | {index}")

        axes[0, column].axis("off")
        axes[1, column].axis("off")

    fig.suptitle(
        f"{dataset_name} - Candidate (orange), Final (green), Rejected (red)\n"
        f"reference slice={reference_index}",
```

```
        fontsize=15,
    )
    plt.show()
```

8.3 Mask-area curve and grow/shrink state

[ ]:
```
fig, axes = plt.subplots(2, 1, figsize=(14, 8), constrained_layout=True)

for axis, (dataset_name, candidates) in zip(axes, candidate_volumes.items()):
    propagated = propagated_volumes[dataset_name]
    candidate_area = candidates.sum(axis=(1, 2))
    propagated_area = propagated.sum(axis=(1, 2))
    reference_index = reference_indices[dataset_name]

    axis.plot(candidate_area, label="Candidate", color="darkorange", alpha=0.8)
    axis.plot(propagated_area, label="Propagated", color="forestgreen")
    axis.axvline(reference_index, color="royalblue", linestyle="--",␣
 ↪label="Reference")
    axis.set_title(f"{dataset_name} - Mask Area per Slice")
    axis.set_xlabel("Slice index")
    axis.set_ylabel("Foreground pixels")
    axis.grid(alpha=0.25)
    axis.legend()

plt.show()

for dataset_name, propagated in propagated_volumes.items():
    area = propagated.sum(axis=(1, 2)).astype(float)
    change = np.diff(area)
    print(
        f"{dataset_name}: {np.count_nonzero(change > 0)} transitions grow, "
        f"{np.count_nonzero(change < 0)} transitions shrink, "
        f"{np.count_nonzero(change == 0)} unchanged"
    )
```

8.4 Axial, coronal, and sagittal views

[ ]:
```
fig, axes = plt.subplots(2, 3, figsize=(16, 10), constrained_layout=True)

for row, (dataset_name, volume) in enumerate(ct_volumes.items()):
    mask = propagated_volumes[dataset_name]
    z = reference_indices[dataset_name]
    y = volume.shape[1] // 2
    x = volume.shape[2] // 3

    views = [
        (volume[z], mask[z], f"Axial | z={z}"),
```

```
        (volume[:, y, :], mask[:, y, :], f"Coronal | y={y}"),
        (volume[:, :, x], mask[:, :, x], f"Sagittal | x={x}"),
    ]

    for axis, (image, view_mask, title) in zip(axes[row], views):
        axis.imshow(image, vmin=LUNG_WINDOW[0], vmax=LUNG_WINDOW[1],␣
↪aspect="auto")
        if view_mask.any():
            axis.contour(view_mask, levels=[0.5], colors="lime", linewidths=1)
        axis.set_title(f"{dataset_name} - {title}")
        axis.axis("off")

plt.show()
```

9 Step 7: Numerical Quality Control

[ ]:
```
def dice_score(mask_a, mask_b):
    denominator = mask_a.sum() + mask_b.sum()
    if denominator == 0:
        return 1.0
    return 2.0 * np.logical_and(mask_a, mask_b).sum() / denominator


qc_results = OrderedDict()

for dataset_name, mask_volume in propagated_volumes.items():
    areas = mask_volume.sum(axis=(1, 2)).astype(float)
    adjacent_dice = np.array([
        dice_score(mask_volume[z - 1], mask_volume[z])
        for z in range(1, len(mask_volume))
    ])
    relative_area_change = np.abs(np.diff(areas)) / np.maximum(areas[:-1], 1.0)
    non_empty = np.flatnonzero(areas > 0)
    largest_jump_indices = np.argsort(relative_area_change)[-5:][::-1] + 1

    qc_results[dataset_name] = {
        "areas": areas,
        "adjacent_dice": adjacent_dice,
        "relative_area_change": relative_area_change,
        "largest_jump_indices": largest_jump_indices,
    }

    print(f"\n{dataset_name}")
    if len(non_empty):
        print(f"  Non-empty range : {non_empty[0]}-{non_empty[-1]}")
    else:
```

```
        print("  Non-empty range : none")
    print(f"  Mean adjacent Dice: {adjacent_dice.mean():.4f}")
    print(f"  Failed-overlap slices: {failed_overlap_slices[dataset_name]}")
    print(f"  Inspect largest area jumps at slices: {largest_jump_indices.
 ↪tolist()}")
```

[ ]:
```
fig, axes = plt.subplots(2, 2, figsize=(15, 8), constrained_layout=True)

for row, (dataset_name, result) in enumerate(qc_results.items()):
    axes[row, 0].plot(np.arange(1, len(result["adjacent_dice"]) + 1),␣
 ↪result["adjacent_dice"])
    axes[row, 0].set_title(f"{dataset_name} - Adjacent-slice Dice")
    axes[row, 0].set_xlabel("Current slice")
    axes[row, 0].set_ylabel("Dice")
    axes[row, 0].set_ylim(0, 1.05)
    axes[row, 0].grid(alpha=0.25)

    axes[row, 1].plot(
        np.arange(1, len(result["relative_area_change"]) + 1),
        result["relative_area_change"],
        color="firebrick",
    )
    axes[row, 1].set_title(f"{dataset_name} - Relative Area Change")
    axes[row, 1].set_xlabel("Current slice")
    axes[row, 1].set_ylabel("Absolute relative change")
    axes[row, 1].grid(alpha=0.25)

plt.show()
```

9.1 Automatically display the slices with the largest area changes

[ ]:
```
for dataset_name, volume in ct_volumes.items():
    indices = qc_results[dataset_name]["largest_jump_indices"]
    masks = propagated_volumes[dataset_name]
    fig, axes = plt.subplots(1, len(indices), figsize=(18, 4),␣
 ↪constrained_layout=True)

    for axis, index in zip(np.atleast_1d(axes), indices):
        axis.imshow(volume[index], vmin=LUNG_WINDOW[0], vmax=LUNG_WINDOW[1])
        if masks[index].any():
            axis.contour(masks[index], levels=[0.5], colors="lime",␣
 ↪linewidths=1.2)
        axis.set_title(f"Slice {index}")
        axis.axis("off")

    fig.suptitle(f"{dataset_name} - Largest Mask-area Changes", fontsize=15)
    plt.show()
```

10 Step 8: Median Filter, Window, Normalize, and Apply the Final Mask

Median filtering is applied slice-wise with size `(1, 3, 3)`, so neighboring axial slices are not mixed. The filtered HU volume is then clipped to the lung window `[-1400, 200]`, normalized to `float32` `[0, 1]`, and set to zero outside the final lung mask.

[ ]:
```
def window_and_normalize(volume, level=WINDOW_LEVEL, width=WINDOW_WIDTH):
    lower = level - width / 2.0
    upper = level + width / 2.0
    clipped = np.clip(volume.astype(np.float32), lower, upper)
    return ((clipped - lower) / (upper - lower)).astype(np.float32)


filtered_volumes = OrderedDict()
normalized_volumes = OrderedDict()
masked_parenchyma_volumes = OrderedDict()

for dataset_name, volume in ct_volumes.items():
    filtered = ndi.median_filter(volume, size=MEDIAN_FILTER_SIZE)
    normalized = window_and_normalize(filtered)
    masked = np.where(
        propagated_volumes[dataset_name],
        normalized,
        np.float32(0.0),
    ).astype(np.float32)

    filtered_volumes[dataset_name] = filtered
    normalized_volumes[dataset_name] = normalized
    masked_parenchyma_volumes[dataset_name] = masked

    print(
        f"{dataset_name}: shape={masked.shape}, dtype={masked.dtype}, "
        f"range=[{masked.min():.1f}, {masked.max():.1f}]"
    )
```

10.1 Median-filter comparison and final lung parenchyma

[ ]:
```
fig, axes = plt.subplots(2, 4, figsize=(19, 9), constrained_layout=True)

for row, (dataset_name, masked_volume) in enumerate(masked_parenchyma_volumes.
 ↪items()):
    index = reference_indices[dataset_name]
    mask = propagated_volumes[dataset_name][index]
    original_volume = ct_volumes[dataset_name]
    normalized_slice = normalized_volumes[dataset_name][index]
    filtered_slice = filtered_volumes[dataset_name][index]
```

```
    axes[row, 0].imshow(original_volume[index], vmin=-1400, vmax=200)
    axes[row, 0].set_title(f"{dataset_name} original HU")
    axes[row, 0].axis("off")

    axes[row, 1].imshow(filtered_slice, vmin=-1400, vmax=200)
    axes[row, 1].set_title("Median filtered (3 x 3)")
    axes[row, 1].axis("off")

    axes[row, 2].imshow(masked_volume[index], vmin=0, vmax=1)
    axes[row, 2].set_title(f"Final parenchyma | slice {index}")
    axes[row, 2].axis("off")

    axes[row, 3].hist(normalized_slice[mask], bins=100, range=(0, 1),␣
 ↪color="steelblue")
    axes[row, 3].set_title("Parenchyma intensity histogram")
    axes[row, 3].set_xlabel("Normalized intensity")
    axes[row, 3].set_ylabel("Pixel count")

plt.show()
```

11 Step 9: Nodule Overlay and Inclusion

11.1 Load LIDC-IDRI consensus nodule records

[ ]:
```
with LIDC_CLUSTER_METADATA_PATH.open(encoding="utf-8-sig", newline="") as file:
    lidc_metadata_rows = list(csv.DictReader(file))

lidc_nodule_records = []
for row in lidc_metadata_rows:
    same_scan = (
        row["patient_id"] == LIDC_PATIENT_ID
        and row["study_instance_uid"] == scan.study_instance_uid
        and row["series_instance_uid"] == scan.series_instance_uid
    )
    if same_scan:
        lidc_nodule_records.append({
            "finding_id": int(row["cluster_id"]),
            "class": row["label"],
            "slice_indices": [
                int(value)
                for value in ast.literal_eval(row["consensus_mask_slice_list"])
            ],
        })

if not lidc_nodule_records:
```

```
    raise ValueError(f"No consensus nodule metadata found for {LIDC_PATIENT_ID}.
 ↪")

lidc_nodule_records.sort(key=lambda record: record["finding_id"])
lidc_scan_folder = (
    f"{LIDC_PATIENT_ID}_{scan.study_instance_uid[-5:]}_"
    f"{scan.series_instance_uid[-5:]}"
)

def lidc_nodule_mask_path(record, slice_index):
    return (
        LIDC_CONSENSUS_MASK_DIR
        / lidc_scan_folder
        / f"cluster_{record['finding_id']}"
        / f"slice_{slice_index}.npy"
    )

print(f"LIDC-IDRI findings: {len(lidc_nodule_records)}")
```

11.2 Load LNDb consensus nodule records

[ ]:
```
lndb_scan_name = LNDB_MHD_PATH.stem
lndb_scan_id = int(lndb_scan_name.split("-")[-1])

with LNDB_CONSENSUS_METADATA_PATH.open(
    encoding="utf-8-sig", newline=""
) as file:
    lndb_metadata_rows = list(csv.DictReader(file))

lndb_nodule_records = []
for row in lndb_metadata_rows:
    if int(float(row["lndbid"])) == lndb_scan_id:
        lndb_nodule_records.append({
            "finding_id": int(float(row["findingid"])),
            "class": row["label"],
            "slice_indices": [
                int(value.strip())
                for value in row["consensus_slice_list"].split(",")
                if value.strip()
            ],
        })

if not lndb_nodule_records:
    raise ValueError(f"No consensus nodule metadata found for {lndb_scan_name}.
 ↪")

lndb_nodule_records.sort(key=lambda record: record["finding_id"])
```

```
def lndb_nodule_mask_path(record, slice_index):
    return (
        LNDB_CONSENSUS_MASK_DIR
        / lndb_scan_name
        / f"finding_{record['finding_id']}"
        / f"slice_{slice_index}.npy"
    )

print(f"LNDb findings: {len(lndb_nodule_records)}")
```

11.3 Calculate inclusion per slice and per finding

[ ]:
```
def calculate_nodule_inclusion(
    dataset_name,
    lung_mask_volume,
    nodule_records,
    mask_path_builder,
):
    rows = []

    for record in nodule_records:
        for slice_index in record["slice_indices"]:
            mask_path = mask_path_builder(record, slice_index)
            if not mask_path.is_file():
                raise FileNotFoundError(f"Nodule mask not found: {mask_path}")

            nodule_mask = np.load(mask_path, allow_pickle=False).astype(bool)
            lung_mask = lung_mask_volume[slice_index]
            if nodule_mask.shape != lung_mask.shape:
                raise ValueError(
                    f"Nodule mask shape {nodule_mask.shape} does not match "
                    f"lung mask shape {lung_mask.shape}: {mask_path}"
                )

            nodule_pixels = int(nodule_mask.sum())
            inside_pixels = int(np.logical_and(nodule_mask, lung_mask).sum())
            inclusion = (
                100.0 * inside_pixels / nodule_pixels
                if nodule_pixels > 0 else np.nan
            )
            rows.append({
                "dataset": dataset_name,
                "finding_id": record["finding_id"],
                "class": record["class"],
                "slice_index": slice_index,
                "inside_pixels": inside_pixels,
```

```
                "nodule_pixels": nodule_pixels,
                "inclusion_percent": inclusion,
            })

    return pd.DataFrame(rows)



lidc_inclusion_before_repair = calculate_nodule_inclusion(
    "LIDC-IDRI",
    protected_lung_volumes["LIDC-IDRI"],
    lidc_nodule_records,
    lidc_nodule_mask_path,
)
lndb_inclusion_before_repair = calculate_nodule_inclusion(
    "LNDb",
    protected_lung_volumes["LNDb"],
    lndb_nodule_records,
    lndb_nodule_mask_path,
)

lidc_inclusion = calculate_nodule_inclusion(
    "LIDC-IDRI",
    propagated_volumes["LIDC-IDRI"],
    lidc_nodule_records,
    lidc_nodule_mask_path,
)
lndb_inclusion = calculate_nodule_inclusion(
    "LNDb",
    propagated_volumes["LNDb"],
    lndb_nodule_records,
    lndb_nodule_mask_path,
)
nodule_inclusion_by_slice = pd.concat(
    [lidc_inclusion, lndb_inclusion], ignore_index=True
)

nodule_inclusion_by_finding = (
    nodule_inclusion_by_slice
    .groupby(["dataset", "finding_id", "class"], as_index=False)
    .agg(
        inside_pixels=("inside_pixels", "sum"),
        nodule_pixels=("nodule_pixels", "sum"),
        annotated_slices=("slice_index", "nunique"),
        minimum_slice_inclusion=("inclusion_percent", "min"),
    )
)
```

```
nodule_inclusion_by_finding["inclusion_percent"] = (
    100.0
    * nodule_inclusion_by_finding["inside_pixels"]
    / nodule_inclusion_by_finding["nodule_pixels"]
)

print("Inclusion per finding")
display(nodule_inclusion_by_finding.round(2))

print("Inclusion per annotated slice")
display(nodule_inclusion_by_slice.round(2))

for dataset_name, group in nodule_inclusion_by_slice.groupby("dataset"):
    overall = 100.0 * group["inside_pixels"].sum() / group["nodule_pixels"].
 ↪sum()
    print(f"{dataset_name} overall pixel-weighted inclusion: {overall:.2f}%")


before_repair = pd.concat(
    [lidc_inclusion_before_repair, lndb_inclusion_before_repair],
    ignore_index=True,
)
after_repair = nodule_inclusion_by_slice.copy()

repair_comparison = []
for dataset_name in after_repair["dataset"].unique():
    before_group = before_repair[before_repair["dataset"] == dataset_name]
    after_group = after_repair[after_repair["dataset"] == dataset_name]
    before_percent = (
        100.0 * before_group["inside_pixels"].sum()
        / before_group["nodule_pixels"].sum()
    )
    after_percent = (
        100.0 * after_group["inside_pixels"].sum()
        / after_group["nodule_pixels"].sum()
    )
    repair_comparison.append({
        "dataset": dataset_name,
        "before_boundary_repair_percent": before_percent,
        "after_boundary_repair_percent": after_percent,
        "improvement_points": after_percent - before_percent,
    })

print("Boundary-repair inclusion comparison")
display(pd.DataFrame(repair_comparison).round(2))
```

11.4 Overlay every annotated nodule slice

[ ]:
```
NODULE_COLORS = [
    "#FF1744", "#00E5FF", "#76FF03", "#D500F9",
    "#FFD600", "#2979FF", "#FF6D00", "#00E676",
]


def show_nodule_inclusion_overlays(
    dataset_name,
    normalized_volume,
    lung_mask_volume,
    nodule_records,
    mask_path_builder,
    inclusion_table,
):
    available_slices = sorted({
        slice_index
        for record in nodule_records
        for slice_index in record["slice_indices"]
    })
    color_by_finding = {
        record["finding_id"]: NODULE_COLORS[index % len(NODULE_COLORS)]
        for index, record in enumerate(nodule_records)
    }
    legend_handles = [
        Patch(
            facecolor=color_by_finding[record["finding_id"]],
            label=f"Finding {record['finding_id']} - {record['class']}",
        )
        for record in nodule_records
    ]

    for page_start in range(0, len(available_slices), NODULE_SLICES_PER_FIGURE):
        page_slices = available_slices[
            page_start:page_start + NODULE_SLICES_PER_FIGURE
        ]
        columns = min(3, len(page_slices))
        rows = (len(page_slices) + columns - 1) // columns
        fig, axes = plt.subplots(
            rows, columns,
            figsize=(5 * columns, 5 * rows),
            constrained_layout=True,
            squeeze=False,
        )

        for axis in axes.ravel():
```

```
            axis.axis("off")

        for axis, slice_index in zip(axes.ravel(), page_slices):
            lung_mask = lung_mask_volume[slice_index]
            parenchyma = np.where(
                lung_mask,
                normalized_volume[slice_index],
                np.float32(0.0),
            )
            axis.imshow(parenchyma, vmin=0, vmax=1)
            if lung_mask.any():
                axis.contour(
                    lung_mask, levels=[0.5], colors="lime", linewidths=1
                )

            title_lines = [f"Slice {slice_index}"]
            for record in nodule_records:
                if slice_index not in record["slice_indices"]:
                    continue

                nodule_mask = np.load(
                    mask_path_builder(record, slice_index),
                    allow_pickle=False,
                ).astype(bool)
                color = color_by_finding[record["finding_id"]]
                rgba = np.zeros((*nodule_mask.shape, 4), dtype=np.float32)
                rgba[nodule_mask] = (*to_rgb(color), 0.65)
                axis.imshow(rgba)
                axis.contour(
                    nodule_mask, levels=[0.5], colors=[color], linewidths=1.5
                )

                row = inclusion_table[
                    (inclusion_table["finding_id"] == record["finding_id"])
                    & (inclusion_table["slice_index"] == slice_index)
                ].iloc[0]
                title_lines.append(
                    f"F{record['finding_id']}: {row['inclusion_percent']:.1f}%"
                )

            axis.set_title(" | ".join(title_lines))

        fig.suptitle(
            f"{dataset_name} - Lung Parenchyma + Nodule Inclusion",
            fontsize=15,
        )
        fig.legend(
```

```
            handles=legend_handles,
            loc="outside lower center",
            ncol=min(4, len(legend_handles)),
        )
        plt.show()


show_nodule_inclusion_overlays(
    "LIDC-IDRI",
    normalized_volumes["LIDC-IDRI"],
    propagated_volumes["LIDC-IDRI"],
    lidc_nodule_records,
    lidc_nodule_mask_path,
    lidc_inclusion,
)
show_nodule_inclusion_overlays(
    "LNDb",
    normalized_volumes["LNDb"],
    propagated_volumes["LNDb"],
    lndb_nodule_records,
    lndb_nodule_mask_path,
    lndb_inclusion,
)
```

12 Step 10: Optional Save

[ ]:
```
SAVE_RESULTS = False
OUTPUT_DIR = Path("outputs/clean_seed_bidirectional_lung_segmentation")

if SAVE_RESULTS:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    for dataset_name in ct_volumes:
        safe_name = dataset_name.lower().replace("-", "_")
        np.save(
            OUTPUT_DIR / f"{safe_name}_lung_mask.npy",
            propagated_volumes[dataset_name].astype(np.uint8),
        )
        np.save(
            OUTPUT_DIR / f"{safe_name}_lung_parenchyma.npy",
            masked_parenchyma_volumes[dataset_name],
        )

    nodule_inclusion_by_slice.to_csv(
        OUTPUT_DIR / "nodule_inclusion_by_slice.csv", index=False
    )
```

```
    nodule_inclusion_by_finding.to_csv(
        OUTPUT_DIR / "nodule_inclusion_by_finding.csv", index=False
    )
    print(f"Saved to: {OUTPUT_DIR.resolve()}")
else:
    print("SAVE_RESULTS=False - no files were written.")
```

13 Interpretation Notes

- Candidate masks use a high-specificity geometric rule to remove wide, flat table components before largest-component selection.
- Small-peripheral cleanup and trachea removal remain restricted to reference/final processing.
- Automatic reference selection prefers the nearest cleaned slice with substantial components on both image halves.
- Component matching protects lung components based on continuity with the preceding clean mask.
- A failed or empty slice does not erase propagation state; the next slice reuses the last valid reference.
- Residual trachea is removed after propagation using a narrow central band.
- Boundary repair is performed per component to prevent the left and right lungs from merging.
- `BOUNDARY_REPAIR_RADIUS=16` is the conservative setting validated for the LIDC-IDRI-0077 juxta-hilar defect.
- Consensus nodule masks are used only to calculate inclusion and do not affect the lung mask.
- Compare inclusion before and after boundary repair and visually inspect the added regions.